

HelixSkills

HelixSkills

Tagline: A governed, constitution-backed skills system for CLI AI agents.

Summary HelixSkills is a skills system for CLI AI agents that inherits the Helix Constitution as a submodule, so every universal governance rule applies unconditionally. It packages installable agent skills, MCP tool servers, Claude Code plugins, and reusable engines behind a registerable, documented catalog.

Short description HelixSkills is a skills system for CLI AI agents. It embeds the Helix Constitution as a submodule so all universal rules apply, then ships registerable skills (action-prefix, media-validator, multitrack, session-sync, workable-item lifecycle, and more), two MCP tool servers, two Claude Code plugins, and reusable engines.

Long description HelixSkills (repo `skills`, Apache-2.0) is a skills system for CLI AI agents, built on a governance-first foundation: it inherits the Helix Constitution as its `constitution/` submodule, so every universal rule from `constitution/CLAUDE.md` and `constitution/Constitution.md` applies unconditionally to any agent using it.

The system ships a concrete, registerable inventory rather than abstractions. Seven constitution skills install via `register.sh`: `action-prefix-system`, `media-validator`, `multitrack`, `reporting-workable-items`, `scheduled-work-queue`, `session-sync`, and `workable-item-lifecycle` (rated intermediate to advanced). Additional draft skills (Android overview, Java/Kotlin language, Linux OS) are indexed and pending activation. Two MCP tool servers (`media-validator`, `scheduled-work`) expose skills to agents over the Model Context Protocol, and two Claude Code plugins (`helix`, `scheduled-work`) integrate with that agent runtime.

Underneath sit four depth-1 reusable engines — `continuum` (implemented), plus `session_orchestrator`, `token_optimizer`, and `clickup_sync` (in design) — with the `token_optimizer` declaring dependencies on vasic-digital ecosystem packages (TOON, Embeddings, VectorDB, Normalize, conversation) and HelixDevelopment's LLMProvider. The project maintains

disciplined documentation: a skills catalog, an auto-generated skill-graph index, per-repo detail pages, and a Gaps & Risks register tracking 136 findings. It is mirrored across GitHub, GitLab, GitFlic, and GitVerse.

Why we built it CLI AI agents need capabilities that are consistent, governed, and reusable — not ad-hoc scripts that each reinvent rules. HelixSkills was built to give agents a packaged, registerable skill set that is bound to a shared constitution, so behavior stays consistent and auditable across every agent and project that adopts it.

Why it's a game-changer It treats agent skills as governed, versioned, installable units backed by a constitution submodule — making agent capabilities portable and rule-compliant by construction, and exposing them through standard surfaces (MCP servers and Claude Code plugins) rather than bespoke glue.

What's innovative - Constitution-as-submodule: universal governance rules are inherited, not copied, so every consuming agent is bound to the same canonical rule set. - Skills delivered as registerable units (`register.sh`) with an auto-generated skill-graph index. - Multi-surface exposure: the same skills reach agents via MCP tool servers and Claude Code plugins. - Reusable depth-1 engines (`continuum`, `token_optimizer`, `session_orchestrator`, `clickup_sync`) shared across the ecosystem, with explicit cross-repo dependency declarations.

Biggest technical challenges + how solved - *Keeping agent behavior consistent and rule-compliant* — solved by inheriting the Helix Constitution as a submodule so rules apply unconditionally rather than being re-implemented per skill. - *Making skills installable and discoverable* — solved with per-skill `register.sh` registration plus an auto-generated INDEX skill graph and per-repo detail docs. - *Exposing skills to different agent runtimes* — solved with MCP tool-server definitions and Claude Code plugin packaging over the same skills.

Tech stack - Shell (primary language): Chosen for portable install/registration tooling; used for `register.sh` and `install_upstreams`. - **Git submodules:** Chosen to inherit governance without duplication; the Helix Constitution is mounted at `constitution/`. - **Model Context Protocol (MCP):** Chosen as the standard agent tool interface; two MCP servers (`media-validator`, `scheduled-work`) are defined under `constitution/mcp/`. - **Claude Code plugins:** Chosen to integrate skills into that agent runtime; two plugins (`helix`, `scheduled-work`) ship under `constitution/plugins/`. - **Reusable engines (`continuum`, `token_optimizer`, `session_orchestrator`, `clickup_sync`):** Chosen for cross-project reuse; `token_optimizer` depends on vasic-digital packages (`TOON`, `Embeddings`, `VectorDB`, `Normalize`,

conversation) and HelixDevelopment's LLMProvider. - **Multi-host Git mirroring (GitHub, GitLab, GitFlic, GitVerse):** Chosen for resilience and regional access.

Public links - Repo: <https://github.com/HelixDevelopment/skills> (public, Apache-2.0). Note: README refers to it as `helix_skills`; the canonical GitHub path is `HelixDevelopment/skills`. - Mirrors (from README): GitLab `helixdevelopment1/helix_skills`, GitFlic `helixdevelopment/helix_skills`, GitVerse `helixdevelopment/helix_skills`. - Related: Helix Constitution — <https://github.com/HelixDevelopment/HelixConstitution> (submodule dependency).

Suggested diagrams/illustrations 1. Governance-inheritance diagram: HelixSkills mounting the Constitution submodule, with universal rules cascading to every registered skill and consuming agent. 2. Skill catalog map: the 7 constitution skills + 4 draft skills grouped by domain/complexity, feeding the auto-generated skill-graph index. 3. Multi-surface exposure: one skill set reaching agents through MCP tool servers and Claude Code plugins. 4. Engine/dependency graph: `token_optimizer` and the other depth-1 engines with their cross-repo dependencies (TOON, Embeddings, VectorDB, Normalize, conversation, LLMProvider).

Site relevance Both — company framing on `vasic.digital` (agent-governance/skills infrastructure) and personal authorship framing on `milosvasic.ru` (Milos as author of the Helix agent-governance model).

Priority tier: Helix-primary

Source provenance - `gh api repos/HelixDevelopment/skills/readme` (primary; fetched live 2026-08-05 — package inventory, constitution skills, draft skills, MCP servers, plugins, engines, dependencies, mirrors, quick start). - `gh repo view HelixDevelopment/skills --json ...` and `gh search repos Skills org:HelixDevelopment` (metadata: public, Apache-2.0, Shell, description "Helix Skills - Skills system for CLI AI Agents", pushedAt 2026-07-24; canonical name `skills`). - Not present in `/Volumes/T7/Projects/vasic/_analysis/github-helix-others.*` harvest (repo absent from the 2026-06-17 inventory — created/renamed later) and not available locally. - UNVERIFIED: the "136 tracked findings (95 open, 40 fixed, 1 N/A)" count is a self-reported figure from the README; engine statuses (design vs implemented) are project-stated.